Processamento de Big Data e Detecção de Ataques em Industrial IoT com Redes Neurais: Estudo Experimental com Edge-IIoTset

RESUMO Este artigo integra dois conjuntos de experimentos realizados com o Edge-IIoTset para investigar, de forma articulada, o armazenamento e o processamento de Big Data e a detecção de ataques cibernéticos em ambientes Industrial Internet of Things (IIoT). O estudo reúne comparações entre CSV e Apache Parquet, pandas e PySpark, além de experimentos com redes neurais Multilayer Perceptron (MLP), regularização por Dropout e otimização do limiar de classificação. A análise considera desempenho computacional, escalabilidade, métricas preditivas e o compromisso entre falsos positivos e falsos negativos. Os resultados devem ser interpretados segundo as condições de execução no Google Colab e as limitações metodológicas registradas nos notebooks.

Palavras-chave: Industrial IoT; Edge-IIoTset; Big Data; Apache Spark; Parquet; MLP; detecção de ataques.

I. INTRODUÇÃO

A segurança de ambientes Industrial Internet of Things (IIoT) envolve tanto a capacidade de armazenar e processar grandes volumes de dados quanto a identificação de comportamentos maliciosos no tráfego de rede. Os dois notebooks que fundamentam este artigo investigam essas dimensões utilizando o Edge-IIoTset. O primeiro examina formatos de armazenamento, mecanismos de execução e desempenho computacional com diferentes volumes de registros. O segundo aborda a classificação binária de tráfego normal e malicioso por meio de redes neurais MLP, comparando arquiteturas e limiares de decisão. O artigo articula os experimentos em uma sequência única, preservando literalmente os trechos selecionados dos notebooks nas seções metodológicas e de resultados.

II. CONJUNTO DE DADOS E PREPARAÇÃO

O conjunto de dados que utilizamos no trabalho apresentado é chamado de Edge-IIoTset, desenvolvido pelo Ferrag et al. (2022), ele tem como objetivo principal apoiar pesquisas relacionadas à segurança cibernética em ambientes de Internet das Coisas (IoT) e Internet Industrial das Coisas (IIoT).

O Edge-IIoTset foi construído a partir de uma infraestrutura experimental que integra dispositivos IoT/IIoT, protocolos de comunicação e diferentes cenários de ataques cibernéticos. Os registros coletados representam características do tráfego de rede e permitem investigar padrões associados a atividades legítimas e maliciosas (Ferrag, 2022).

O conjunto de dados foi escolhido por se adequar de forma satisfatoria aos objetivos expecificos da pesquisa, uma vez que nos fornece informações que podem ser utilizadas tanto em experimentos de processamento de Big Data quanto no desenvolvimento de modelos de detecção de anomalias baseados em redes neurais.

Um outro pontom possitivo relacionado a base de dados é que se trata de uma base dados publica, sendo encontrada publicamente no Kaggle, o que viabializa a sua utilização em experimentos acadêmicos.

O arquivo original contém um total de 2.219.201 registros e 63 colunas. Como identificamos nos testes que foram realizados a baixo.

Apartir da identificacao desses valores pudemos iniciar a investigação de estrategias de de armazenamento, gerenciamento e processamento de dados em diferentes escalas.

Um ponto forte do trabalho é que foi possivel realizar diferentes experimentos com o numero diferente de registros e avaliar como as tecnologias se comportam á medida que a quantidade de dados aumenta.

Com isso buscamos tratar 3 conceitos principais:

Armazenamento -> Comparamos como os formatos CSV e Parquet ocupam e quanto tempo levam para ler e escrever dados

Processamento -> Comparamos os tempos de execução de pandas e PySpark conforme aumentamos o número de registros.

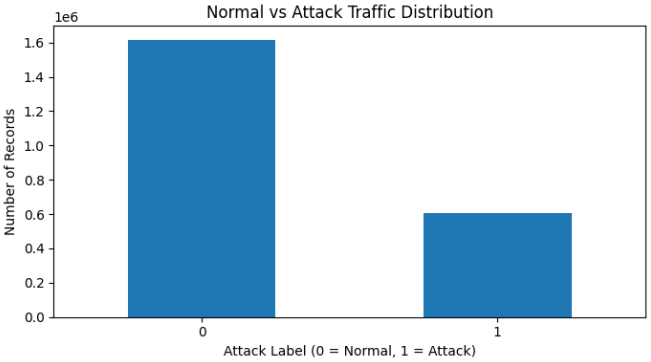


Fig. 1. Distribuição das classes no Edge-IIoTset. Fonte: gráficos e diagramas originais dos notebooks.

No projeto desenvolvido, o dataset que selecionamos tem aproximadamente 2,2 milhões de registros, apartir dele selecionamos amostras de tamanhos variados para a realização dos nosso experimentos, sendo as sequintes:

- 200.000 registros: os registros foram selecionados com o intuito de avaliar o desempenho com um volume menor de dados.

- 500.000 registros: selecionamos os registros com o intuito de observar o comportamento com um volume intermediário de dados.

- 1.000.000 de registros: selecionamos essa quantidade de registros com o intuito de investigar o impacto causado por um volume maior no tempo de processamento total.

O dataset selecionado contem 63 colunas, que incluem atributos relacionados ao tráfego de rede, informações sobre protocolos e variáveis utilizadas para identificar atividades maliciosas(Ferrag, 2022).

Durante os processos exploratorios de teste foi observado que o conjuto de dados contém duas variáveis de classificação que se mostraram bastante promissoras, uma vez que elas nos fornecem informações sobre a presenca e a natureza de atividades malicionas no trafico de rede, sendo elas:

- Attack_label: identifica se o registro representa tráfego normal com a lable 0 ou malicioso com a lable 1.

Outros trabalhos exploraram essas duas variaveis como na pesquisa realizada por Ferrag et al. (2022), responsáveis pela criação do Edge-IIoTset, que desenvolveram os dois rótulos para apoiar o desenvolvimento e a avaliação de sistemas de detecção de intrusões baseados em aprendizagem de máquina.

Alguns estudos posteriores também exploraram essa estrutura de classificação. Um exemplo interresante encontra-se no artigo A deep learning approach for multiclass attack classification in IoT and IIoT networks using convolutional neural networks (2025) que investigou diferentes níveis de classificação, incluindo detecção binária e identificação de categorias específicas de ataques.

Com o auxilio dessas informações pudemos destacar essas duas variaveis como sendo extremamente importantes para o trabalho que estamos realizando, uma vez que possivelmente atravez delas é possivel desenvolver e analisar modelos de detecção binarias de anomalias, nos

permitindo identificar diferentes tipos de ataques e identificar modelos de classificação multi-classes.

O que buscamos ao realizar a distinção dessas duas variaveis é compreender a presenca de atividade maliciosa e a sua diversidade, buscando fornecendo uma base solida para a preparação dos dados e para os experimentos posteriores de detecção de anomalias.

Uma importante observação -> A variável Attack_label foi utilizada para realizar a amostragem estratificada e preservar a distribuição das classes durante os experimentos.

Após a aquisição e a verificação do arquivo DNN-EdgeIIoT-dataset.csv, foi realizada uma análise exploratória inicial utilizando a biblioteca pandas, com o objetivo de compreender a estrutura e as características dos dados disponíveis.

Para esta etapa, foram carregados os primeiros 100.000 registros, correspondentes a uma amostra inicial do arquivo original, que contém aproximadamente 2,2 milhões de observações.

A dimensão (100000, 63) indica que o DataFrame contém 100.000 linhas e 63 colunas. uma observação importante é que o consumo de memória de 102,52 MB corresponde a forma que o ambiente Python representa os dados, e não ao tamanho do arquivo CSV original.

A visualização abaixo dos cinco primeiros registros permitiu identificar atributos associados a diferentes protocolos de comunicação,sendo alguns deles incluindo ARP, ICMP e MQTT, além de informações como endereços IP de origem, destino e registros temporais.

ARP, ICMP e MQTT são protocolos de comunicação utilizados em redes de computadores e dispositivos IoT (Internet das Coisas). Cada um desempenha uma função diferente na comunicação entre dispositivos. (MINOLI, 2019)

No nosso projeto com o dataset Edge-IIoTset, esses protocolos são importantes porque representam diferentes características do tráfego de rede que podem ser analisadas para identificar comportamentos normais e possíveis ataques cibernéticos.

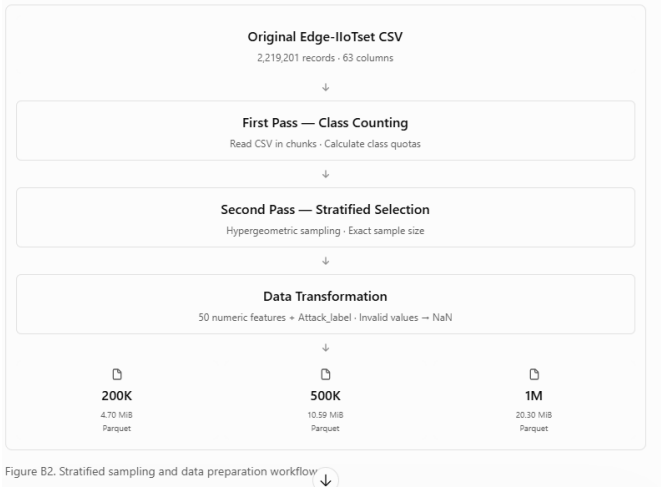


Fig. 2. Procedimento de amostragem estratificada e preparação dos dados. Fonte: gráficos e diagramas originais dos notebooks.

Um ponto importante observado é que o consumo de memória observado mostra a importância de considerar os recursos computacionais disponíveis durante a excução dos experimentos.

Em síntese, a exploração inicial confirmou a estrutura tabular do Edge-IIoTset e nos permitiu identificar a diversidade dos atributos que estão presentes no data set. Com essas informações conseguimeos desenvolver uma base solida para as etapas seguintes que incluem:

- A análise dos tipos de dados

- A distribuição das classes

- Seleção de atributos

- Amostragem estratificada.

Após a exploração inicial do Edge-IIoTset, realizamos a análise da estrutura do DataFrame utilizando a função info() da biblioteca pandas, com o objetivo de identifica:

Os resultados nos demonstram a predominância de atributos numéricos, eles representam aproximadamente 81% das colunas, enquanto os demais atributos correspondem a informações textuais, como endereços IP, identificadores de protocolos e categorias de ataques.

A análise também identificou a auxencia de valores nulos nas 100.000 selecionadas. Entretanto, essa observação não garante a inexistência de valores inválidos, infinitos ou ausentes em outras partes do dataset.

III. ARMAZENAMENTO E PROCESSAMENTO DE BIG DATA

Após a exploração inicial e a identificação dos atributos numéricos do Edge-IIoTset, foi implementada uma estratégia de amostragem estratificada para preparar conjuntos de dados representativos destinados aos experimentos de armazenamento e processamento.

A necessidade dessa abordagem foi identificada durante o Experimento B1, quando se observou que os primeiros 100.000 registros do arquivo original pertenciam exclusivamente à classe de tráfego normal. Entretanto, a análise completa do dataset demonstrou uma distribuição de aproximadamente 72,8% de registros normais e 27,2% de registros maliciosos.

Essa diferença evidenciou que a seleção sequencial dos primeiros registros poderia produzir amostras não representativas. Portanto, optou-se por utilizar a variável Attack_label como critério de estratificação, preservando aproximadamente a proporção original entre as duas classes.

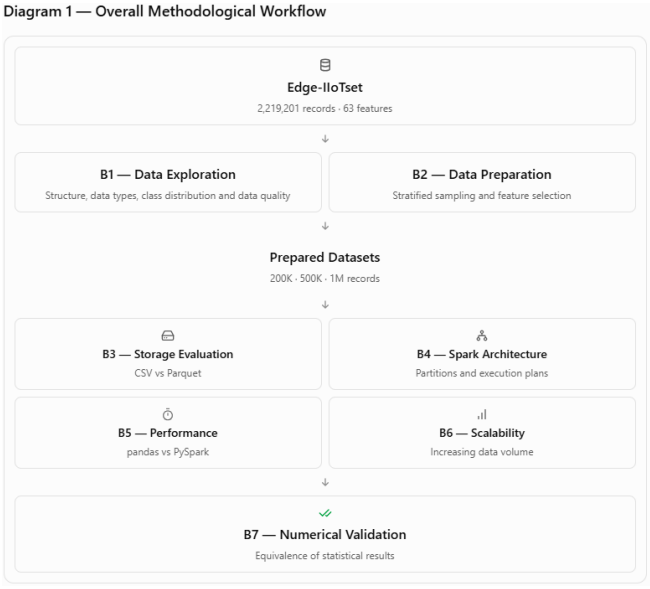


Fig. 3. Fluxo metodológico dos experimentos de Big Data. Fonte: gráficos e diagramas originais dos notebooks.

O procedimento foi implementado por meio da função stratified_csv_sample(), desenvolvida em Python para realizar a amostragem sem carregar integralmente o arquivo CSV na memória.

A função utiliza uma estratégia de leitura em blocos (chunk-based processing), com blocos de até 100.000 registros. Inicialmente, o arquivo é percorrido para contabilizar a quantidade de registros pertencentes a cada classe. Em seguida, são calculadas as quotas

proporcionais necessárias para produzir uma amostra com o tamanho solicitado.

Em uma segunda passagem pelo arquivo, os registros são selecionados aleatoriamente dentro de cada classe, utilizando uma distribuição hipergeométrica para controlar a quantidade de observações selecionadas em cada bloco. Essa estratégia permite construir uma amostra de tamanho exato, respeitando as quotas estabelecidas para as classes.

Após a amostragem, os atributos selecionados no Experimento B1 foram convertidos para formatos numéricos. Valores incompatíveis com essa representação foram transformados em valores ausentes (NaN), assim como eventuais valores infinitos.

A variável Attack_label foi mantida separadamente como informação de classificação, enquanto os 50 atributos numéricos foram preservados para as operações analíticas posteriores.

O procedimento produziu inicialmente uma amostra de 1.000.000 de registros, utilizada como base para gerar três conjuntos experimentais:

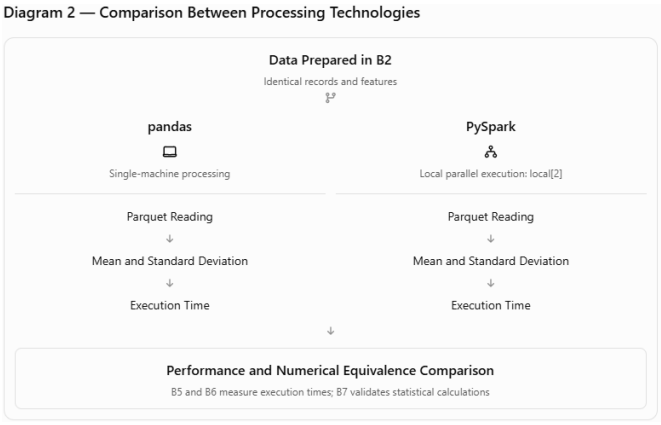


Fig. 4. Comparação das tecnologias de processamento. Fonte: gráficos e diagramas originais dos notebooks.

A utilização de diferentes volumes de registros possibilita investigar como o aumento da quantidade de dados influencia os requisitos de armazenamento e o desempenho computacional.

Além disso, a preparação de conjuntos com a mesma estrutura de atributos permite realizar comparações consistentes entre pandas e PySpark, reduzindo diferenças metodológicas que poderiam comprometer a interpretação dos resultados.

Após a preparação dos conjuntos de dados no Experimento B2, foi realizada uma avaliação comparativa dos formatos CSV (Comma-Separated Values) e Apache Parquet, com o objetivo de investigar o impacto da estrutura de armazenamento sobre a utilização de espaço em disco e o desempenho das operações de entrada e saída de dados.

O CSV é um formato textual amplamente utilizado para armazenamento e intercâmbio de dados tabulares. Sua principal vantagem está na simplicidade, na portabilidade e na compatibilidade com diferentes ferramentas. Entretanto, sua representação textual pode aumentar o espaço necessário para armazenar valores numéricos e exigir operações adicionais de interpretação e conversão durante a leitura.

O Apache Parquet, por sua vez, utiliza armazenamento colunar, organizando os valores de acordo com os atributos da tabela. Essa estrutura favorece mecanismos de codificação e compressão, além de permitir que ferramentas analíticas recuperem apenas as colunas necessárias para determinadas operações. Por essas características, o Parquet é frequentemente utilizado em arquiteturas de processamento analítico de grandes volumes de dados (Apache Parquet, s.d.).

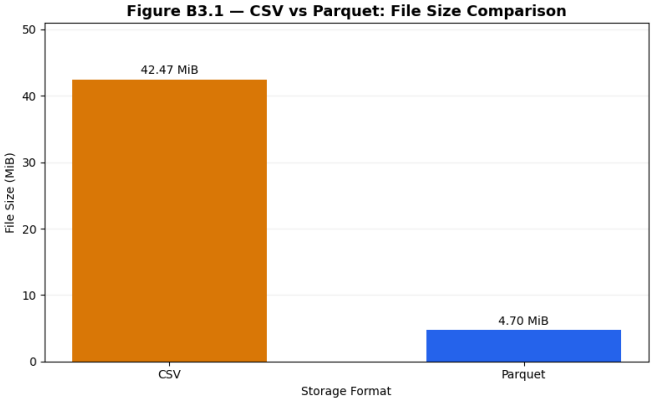


Fig. 5. Comparação do tamanho dos arquivos CSV e Parquet. Fonte: gráficos e diagramas originais dos notebooks.

Para comparar os dois formatos, foi utilizado o conjunto de **200.000 registros e 51 colunas** preparado no Experimento B2, contendo 50 atributos numéricos e a variável de classificação Attack_label.

1. **Espaço de armazenamento:** medição do tamanho dos arquivos CSV e Parquet, expresso em MiB.

2. **Tempo de escrita:** medição do tempo necessário para gravar o mesmo conjunto de dados em cada formato.

3. **Tempo de leitura:** medição do tempo necessário para carregar os arquivos, utilizando cinco execuções cronometradas após uma leitura inicial de aquecimento (warm-up).

As medições foram realizadas com a função time.perf_counter(), apropriada para medir intervalos de execução com um relógio de alta resolução.

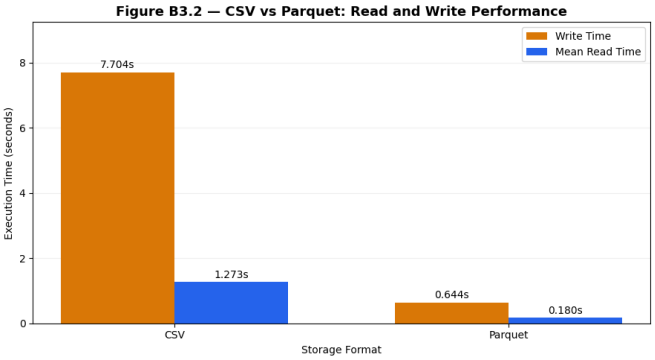


Fig. 6. Tempos de leitura e escrita dos formatos CSV e Parquet. Fonte: gráficos e diagramas originais dos notebooks.

Para os tempos de leitura, foram calculadas a média e o desvio-padrão das cinco execuções, permitindo avaliar tanto o desempenho médio quanto a variabilidade observada.

Os resultados foram organizados em um DataFrame e exportados para um arquivo CSV, possibilitando sua utilização posterior na análise comparativa.

É importante destacar que esse experimento avalia operações de leitura e escrita realizadas com pandas no ambiente Google Colab. Portanto, os resultados não representam uma comparação direta entre mecanismos distribuídos de armazenamento, mas uma avaliação experimental dos formatos de arquivo sob condições locais.

A execução do Experimento B3 permitiu comparar o espaço de armazenamento e os tempos de leitura e escrita dos formatos CSV e Apache Parquet, utilizando o mesmo conjunto de 200.000 registros e 51 colunas.

Os resultados demonstram que o formato Apache Parquet apresentou menor utilização de espaço de armazenamento e tempos inferiores de leitura e escrita nas condições avaliadas.

O arquivo CSV ocupou aproximadamente 42,47 MiB, enquanto o arquivo Parquet ocupou 4,70 MiB, representando uma redução de aproximadamente **88,94% no espaço utilizado**.

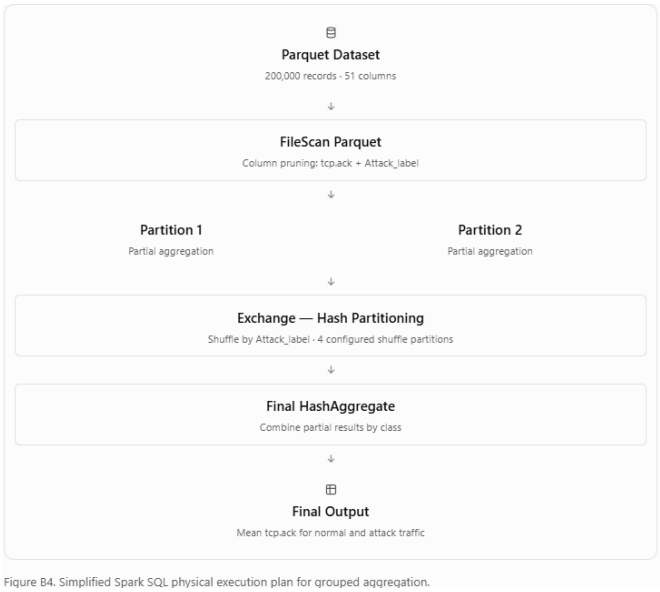


Figure B4. Simplified Spark SQL physical execution plan for grouped aggregation.

Fig. 7. Plano físico de execução Spark SQL. Fonte: gráficos e diagramas originais dos notebooks.

Essa diferença pode ser explicada pela representação textual dos valores no CSV e pelos mecanismos de codificação e compressão utilizados pelo formato colunar Parquet.

Nesse experimento, a gravação em Parquet foi aproximadamente **11 vezes mais rápida** que a gravação em CSV.

Esse resultado demonstra uma vantagem do Parquet nas condições testadas. Entretanto, como foi realizada apenas uma medição de escrita para cada formato, não é possível avaliar estatisticamente a variabilidade desses tempos.

O desvio-padrão também foi menor para o Parquet, indicando menor variação absoluta dos tempos de leitura entre as cinco execuções observadas.

Entretanto, como as leituras foram realizadas após uma etapa de aquecimento, os resultados podem ter sido influenciados pelo cache do sistema operacional e pelas condições momentâneas do ambiente Google Colab.

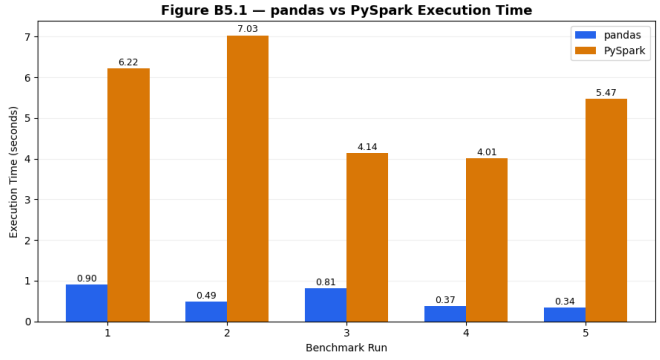


Fig. 8. Comparação dos tempos de execução pandas e PySpark. Fonte: gráficos e diagramas originais dos notebooks.

De maneira geral, pudemos constatar que apartir dos resultados que o Apache Parquet foi mais eficiente que o CSV para armazenar e recuperar o conjunto numérico utilizado neste experimento.

A redução do tamanho dos arquivos pode contribuir para diminuir requisitos de armazenamento e transferência de dados, enquanto os menores tempos de leitura podem beneficiar operações analíticas repetitivas.

Essas características justificam a utilização do Parquet nos experimentos posteriores com pandas e PySpark, permitindo que ambas as tecnologias trabalhem com uma representação de dados consistente.

É importante resaltar que os resultados não demonstram que o Parquet será necessariamente superior em todos os cenários. O desempenho depende de fatores como tipos de dados, compressão, hardware, biblioteca utilizada e características da carga de trabalho.

Portanto, o Experimento B3 nos fornece evidências de que, considerando o conjunto de dados numérico selecionado para experimento e para o ambiente computacional avaliado no momento, o Apache Parquet apresentou vantagens de armazenamento e desempenho em relação ao CSV.

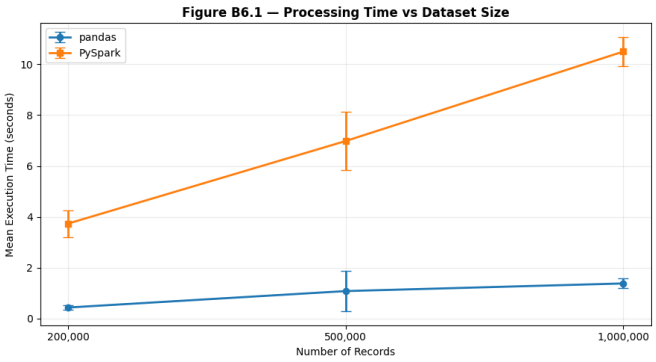


Fig. 9. Escalabilidade do tempo de processamento por volume. Fonte: gráficos e diagramas originais dos notebooks.

Após a avaliação dos formatos de armazenamento no Experimento B3, foi realizada uma investigação dos mecanismos de processamento do Apache Spark, com o objetivo de compreender como o framework organiza e executa operações analíticas sobre dados estruturados.

O Apache Spark é um mecanismo de processamento de dados que oferece suporte à execução paralela e distribuída. Sua arquitetura permite dividir operações em tarefas, processar partições de dados e combinar resultados por meio de mecanismos de agregação e comunicação entre processos (Zaharia et al., 2016).

Uma característica importante do Spark é a utilização do Spark SQL, que permite executar operações sobre dados estruturados por meio da API DataFrame. O Spark SQL incorpora mecanismos de otimização de consultas e seleção de planos de execução, permitindo reduzir operações desnecessárias e organizar o processamento de maneira eficiente (Armbrust et al., 2015).

Neste experimento, foi utilizado o conjunto de dados contendo **200.000 registros e 51 colunas**, preparado anteriormente no formato Apache Parquet.

O ambiente Spark foi configurado no Google Colab utilizando o modo de execução paralela local local[2], que disponibiliza duas threads de execução para o processamento das tarefas.

O procedimento experimental envolveu a leitura do arquivo Parquet, a identificação do número de partições e a execução de uma operação de agregação estatística.

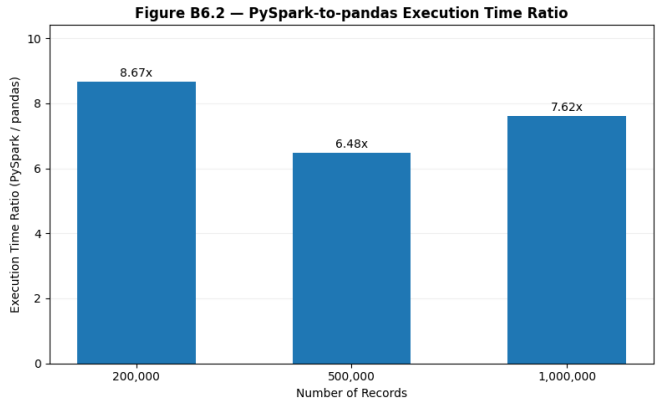


Fig. 10. Razão entre os tempos de execução PySpark e pandas. Fonte: gráficos e diagramas originais dos notebooks.

Para essa análise, os registros foram agrupados de acordo com a variável Attack_label, calculando-se a média do atributo numérico tcp.ack para cada classe.

No plano lógico otimizado, foi identificada a operação Project, responsável por selecionar os atributos necessários ao processamento.

IV. REDES NEURAIS E DETECÇÃO DE ATAQUES

Temos como objetivo nesta etapa implementar um procedimento de amostragem aleatória estratificada que permita selecionar um um conjunto de amostras significativos no dataset Edge-IIoTset, buscamos preservar a proporção original entre tráfego normal e malicioso, o que buscando com isso é evitar o carregamento simultâneo de todo o arquivo CSV na memória.

Como ja mensionamos anteriormente o Edge-IIoTset foi desenvolvido por Ferrag et al. (2022) para apoiar investigações sobre segurança cibernética em ambientes IoT e Industrial IoT. O conjunto de dados presentes nesse dataset representa diferentes categorias de ataques e nos disponibiliza a variável Attack_label, com ela é possivel distinguir os registros de tráfego normal (0) e malicioso (1).

1. **Visualizar a representatividade das classes:** com a seleção dos primeiros registros do arquivo pode introduzir viés quando os dados apresentam ordenação por categoria ou tipo de tráfego.

2. **Controle dos recursos computacionais:** a seleção de um subconjunto permite reduzir a memória necessária para os experimentos de treinamento.

3. **Consistência experimental:** a utilização da mesma amostra nos experimentos E1, E2 e E3 permite comparar modelos sob condições equivalentes.

A importância da estratificação na avaliação de classificadores é discutida por Kohavi (1995), que investigou estratégias de validação e estimativa de desempenho. Embora o autor tenha estudado principalmente métodos de validação cruzada, sua pesquisa fornece suporte conceitual à preservação da composição das classes.

Para implementar essa estratégia, desenvolvemos uma função chamada de stratified_csv_sample(), essa função combina :

- leitura incremental com pandas

- cálculo proporcional de quotas

- seleção aleatória sem reposição baseada na distribuição hipergeométrica.

- seed - O parametro permite controlar a sequência de números gerados, contribuindo para a reprodutibilidade da seleção.

A definição de uma semente fixa é importante porque os experimentos posteriores dependem da composição da amostra selecionada. É importante salientar que embora a seleção seja aleatoria eles seguem uma sequencia determinada pela semente, ou seja, podemos testar diferentes arquiteturas mais essas arquiteturas serão treinadas com as mesmas amostras, o que nos da uma base de comparação mais acertiva.

- Se cada uma das arquiteturas fossem treinadas com uma amostras diferentes, seria mais difícil determinar se as diferenças de desempenho ocorreram devido à arquitetura ou às características dos registros selecionados

Essa decisão está relacionada à necessidade de consistência na avaliação experimental de classificadores, discutida por Kohavi (1995).

Entretanto, Kohavi não estabelece o valor 42 como padrão. A escolha específica da semente é uma decisão de implementação deste projeto.

O objetivo desse bloco de codigo é determinar a quantidade de registros pertencentes a cada classe no conjunto de dados original:

- usecols=[label] restringe a leitura à variável de classificação, evitando carregar atributos que não são necessários durante essa etapa.

- value_counts() contabiliza os registros de cada classe dentro do bloco atual, enquanto counts.add() acumula as contagens ao longo das sucessivas leituras.

- fill_value=0 permite combinar as contagens mesmo quando determinada classe não está presente em um dos blocos.

A leitura incremental foi selecionada para reduzir o consumo de memória durante a análise de um arquivo de grande volume.

Essa abordagem permite determinar a distribuição global das classes sem armazenar simultaneamente todas as observações do CSV.

Apos contabilizar o numero total de registros contidos em cada classe (Normal e Attack), com isso é possivel identificar o numero total de distribuições disponiveis no dataset.

- Essa informação é necessária para calcular a proporção de cada classe e definir quantos registros deverão ser selecionados durante o procedimento de amostragem estratificada.

Inicialmente, o algoritmo calcula a quantidade esperada de registros por classe, multiplicando sua frequência original pela fração total de amostragem.

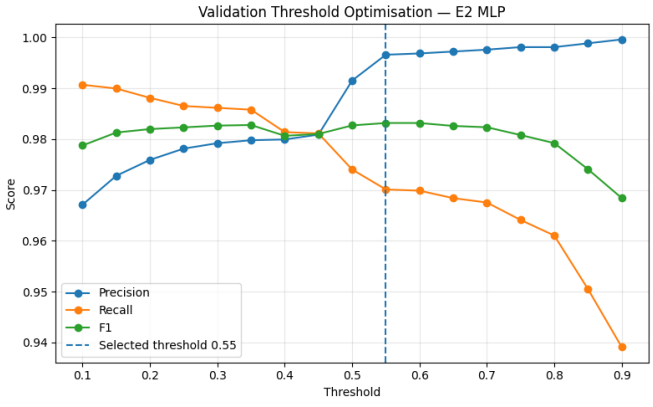


Fig. 11. Otimização do limiar de decisão no conjunto de validação. Fonte: gráficos e diagramas originais dos notebooks.

- \(E_c\) representa a quantidade esperada de registros da classe.

- \(N_c\) representa a quantidade original de registros da classe.

- \(n\) representa o tamanho desejado da amostra.

- \(N\) representa o número total de registros do dataset.

Como o número de registros selecionados deve ser inteiro, utiliza-se np.floor() para arredondar inicialmente as quotas para

baixo.

Por esse motivo, o algoritmo calcula a diferença por meio da variável remainder e distribui os registros restantes entre as classes que apresentaram as maiores partes fracionárias.

A utilização de quotas proporcionais foi adotada para preservar aproximadamente a distribuição original entre tráfego normal e malicioso.

Essas estruturas são atualizadas durante a segunda passagem pelo arquivo, permitindo que o algoritmo considere as seleções realizadas anteriormente.

- pop: número de registros restantes daquela classe no conjunto ainda não processado.

- quota: número de registros daquela classe que ainda precisam ser selecionados.

- available: número de registros da classe disponíveis no bloco atual.

Essa distribuição representa o número de elementos de determinada categoria obtidos em uma amostra aleatória retirada de uma população finita, sem reposição.

- \(N\): número de registros restantes da classe.

- \(K\): quantidade de registros ainda necessária para completar a quota.

- \(n\): número de registros da classe disponíveis no bloco.

- \(k\): quantidade sorteada para seleção.

O mecanismo permite distribuir a seleção aleatória entre os blocos sem estabelecer uma quantidade fixa de registros por bloco.

O problema mais amplo da seleção aleatória sem reposição em grandes conjuntos de dados também é investigado por Vitter (1985).

É importante destacar que a implementação deste notebook não utiliza o algoritmo de reservoir sampling proposto por Vitter. O código combina uma estratégia de duas passagens com sorteios hipergeométricos e seleção aleatória dentro de cada bloco.

Após determinar a quantidade de registros a selecionar, o algoritmo utiliza group.sample() para escolher aleatoriamente as observações correspondentes.

A utilização de random_state, derivado do gerador de números aleatórios inicializado anteriormente, permite controlar a aleatoriedade do procedimento.

A variável remaining_population é reduzida pelo número de registros processados no bloco, enquanto remaining_quota é reduzida pelo número efetivamente selecionado.

Entretanto, ela não comprova, isoladamente, que a distribuição das classes foi preservada ou que a seleção foi estatisticamente representativa.

O valor foi selecionado como uma configuração prática para controlar o consumo de memória, mas não representa um tamanho ótimo demonstrado experimentalmente.

A implementação da amostragem estratificada representa uma etapa essencial para a preparação dos experimentos de redes neurais.

Ao preservar aproximadamente a distribuição original entre tráfego normal e malicioso, o procedimento reduz o risco de distorções introduzidas por uma seleção sequencial de registros.

A utilização de uma amostra de tamanho controlado também permite adequar os experimentos às limitações computacionais do Google Colab.

Além disso, o uso da mesma amostra nos experimentos E1, E2 e E3 contribui para uma comparação mais consistente entre diferentes arquiteturas neurais e estratégias de classificação.

O objetivo desta seção:

- definir o problema de classificação que sera explorado

- justificar a seleção da arquitetura neural

- extabelecar um conjunto de dados apropriado para os experimentos posteriores de detecção de tráfego malicioso.

O dataset Edge-IIoTset, originalmente desenvolvido por Ferrag et al. (2022), foi desenvolvido com o intuito de investigar a capacidade de redes neurais artificiais de distinguir comportamentos normais e maliciosos em ambientes Industrial Internet of Things (IIoT).

Fundamentamos a nossa escolha considerando a estrutura original do Edge-IIoTset, que já nos disponibiliza a variável Attack_label especificamente para essa finalidade (Ferrag et al., 2022).

Dessa forma, buscamos desenvolver um modelo capaz de aprender relações entre características numéricas do tráfego de rede e a presença de atividades maliciosas.

COnsideranto tudo que já foi discutido anteriormente definimos como arquitetura inicial a Multilayer Perceptron (MLP), que pertencente à família das redes neurais artificiais do tipo feedforward.

Uma MLP é composta por uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. Durante o treinamento, os parâmetros são ajustados para aprender relações entre os atributos de entrada e os valores esperados da classificação. (Scikit-learn developers, 2026)

Segundo Goodfellow, Bengio e Courville (2016), as redes neurais feedforward podem aprender funções não lineares por meio da composição de múltiplas transformações, permitindo representar relações complexas entre variáveis.

No trabalho que estamos propondo, a seleção da MLP foi motivada pela utilização de atributos numéricos tabulares de comprimento fixo presentes no dataset que estamos utilizando como base, extraídos do tráfego de rede.

Essa representação permite utilizar cada registro como um vetor de características, sem exigir a construção prévia de sequências temporais ou estruturas espaciais.

Ferrag et al. (2022) também investigaram modelos de aprendizagem profunda utilizando o Edge-IIoTset, fornecendo uma referência diretamente relacionada à aplicação de redes neurais para detecção de intrusões.

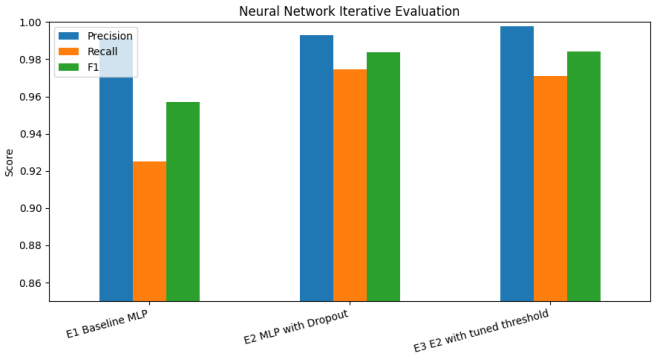


Fig. 12. Comparação de precision, recall e F1 nos experimentos E1, E2 e E3. Fonte: gráficos e diagramas originais dos notebooks.

Embora tenhamos selecionado a MLP para os experimentos E1 e E2, outras arquiteturas poderiam ser facilmente consideradas. São elas:

Convolutional Neural Networks (CNN): utilizam operações convolucionais para explorar relações locais(Google Cloud,

2026).Sua aplicação aos atributos tabulares do Edge-IIoTset exigiria justificar uma organização dos atributos na qual a proximidade entre posições possuísse significado relevante, o que nao ocorre nos atributos descritos no Edge-IIoTset que são variáveis tabulares que não possuem necessariamente uma relação de proximidade significativa.

É importante resaltar que a utilização de CNNs em dados tabulares é possível, mas não foi investigada experimentalmente neste notebook.

Recurrent Neural Networks (RNN), LSTM e GRU: são arquiteturas adequadas para modelar dependências sequenciais(Coursera, 2026). Poderiam ser utilizadas caso os registros fossem organizados em sequências temporais válidas, preservando informações sobre a evolução do tráfego. Entretanto, a representação utilizada neste estudo considera registros individuais, sem construir explicitamente essas sequências.

Transformers: podem modelar relações entre elementos de sequências por mecanismos de atenção(IBM, 2025). Sua utilização exigiria definir uma representação apropriada dos dados e avaliar se os benefícios justificariam a complexidade adicional.

Autoencoders: podem aprender representações comprimidas dos dados e ser utilizados em estratégias de detecção de anomalias baseadas em erros de reconstrução(DataCamp, 2025). Representam uma alternativa relevante para cenários com disponibilidade limitada de rótulos, mas não foram implementados nos experimentos apresentados.

É importante resaltar que o fato de termos selecionado a MLP para os experimentos não implica que els é superior as outras redes neurais citadas a cima. Trata-se de uma escolha metodológica que consideta o dataset utilizado, o formato dos atributos e aos objetivos da investigação.

- nrows=10_000 limita a quantidade de registros carregados, permitindo uma inspeção inicial da estrutura do dataset sem realizar sua leitura integral.

- É importante resaltar que o valor de 10.000 registros representa uma escolha prática uma vez que estamos realizando apenas uma inspeção preliminar, não coresponde ao tamanho amostral estatisticamente otimizado.

- assert a variavel verifica se as variáveis Attack_label e Attack_type estão presentes no conjunto de dados.

- Essa validação foi incluída porque essas variáveis possuem funções específicas na estrutura do Edge-IIoTset, conforme descrito por Ferrag et al. (2022).

- Attack_label utilizamos a varivel como referência para a classificação binária, enquanto Attack_type representa a categoria específica do tráfego ou ataque.

A realização desta verificação nos ajuda a evita que as etapas posteriores sejam executadas em um arquivo que não contem os rótulos esperados.

* Entretanto, a presença das colunas não garante, isoladamente, que seus valores estejam corretos. Por esse motivo, a variável-alvo também será submetida a uma conversão numérica na etapa seguinte.

O codigo acima foi desenvolvido com o intuito de identificar os atributos numéricos que poderão ser utilizados como entradas dos modelos neurais.

- difference() exclui as variáveis Attack_label e Attack_type da lista de atributos preditores.

- Essa exclusão é fundamental porque ambas representam informações relacionadas à classificação.

- Se incliossemos a variavel Attack_label como entrada, essa ação permitiria que o modelo acessasse diretamente a resposta que deveria aprender a prever.

* Portanto, as duas variáveis devem ser excluídas das entradas do modelo.

A separação entre atributos preditores e variável-alvo está relacionada aos fundamentos da aprendizagem supervisionada discutidos por Goodfellow, Bengio e Courville (2016).

Esse resultado no mostra que foram selecionaos **50 atributos numéricos candidatos** com o intuito de representar cada registro de tráfego de rede.

- Esses atributos constituem a base inicial para a construção dos vetores de entrada da MLP.

A preservação da composição das classes também está relacionada aos princípios de avaliação de classificadores discutidos por Kohavi (1995).

V. REFERÊNCIAS (transcritas dos notebooks)

Ferrag, Mohamed Amine, et al. "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning." IEEe Access 10 (2022): 40281-40306.

Seyedkolaei, Ali Abdi, Fatemeh Mahmoudi, and José García. "A deep learning approach for multiclass attack classification in IoT and IIoT networks using convolutional neural networks." Future Internet 17.6 (2025): 230. -< https://www.mdpi.com/1999-5903/17/6/230

McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, pp. 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Zaharia, M., Xin, R.S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M.J., Ghodsi, A., Gonzalez, J., Shenker, S. and Stoica, I. (2016) 'Apache Spark: A Unified Engine for Big Data Processing', Communications of the ACM, 59(11), pp. 56–65. https://doi.org/10.1145/2934664

Goldberg, D. (1991) 'What Every Computer Scientist Should Know About Floating-Point Arithmetic', ACM Computing Surveys, 23(1), pp. 5–48. https://doi.org/10.1145/103162.103163

Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L. and Janicke, H. (2022) 'Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning', IEEE Access, 10, pp. 40281–40306. Available at: https://doi.org/10.1109/ACCESS.2022.3165809

McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, pp. 56–61. Available at: https://doi.org/10.25080/Majora-92bf1922-00a

Kohavi, R. (1995) 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection', Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1145. Available at: Artigo completo em PDF

Vitter, J.S. (1985) 'Random Sampling with a Reservoir', ACM Transactions on Mathematical Software, 11(1), pp. 37–57. Available at: https://doi.org/10.1145/3147.3165

Pandas development team (2026) pandas.read_csv — pandas documentation. Available at: pandas.read_csv documentation

Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L. and Janicke, H. (2022) 'Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning', IEEE Access, 10, pp. 40281–40306. https://doi.org/10.1109/ACCESS.2022.3165809

Kohavi, R. (1995) 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection', Proceedings of the 14th International Joint Conference on Artificial Intelligence, pp. 1137–1145. Available at: Artigo completo em PDF